Week 3:Scatter, Bar and Histogram

Simple is better than complex. Now is better than never.

https://plotly.com/python/reference

plotly | Graphing Libraries

Python Forum Pricing Dash Star 10,960

Search...

Trace Types

Simple
Scatter
Scatter GL
Bar
Pie
Heatmap
Heatmap GL
Image
Contour

plotly
Join the Plotly Team!
View Open Positions

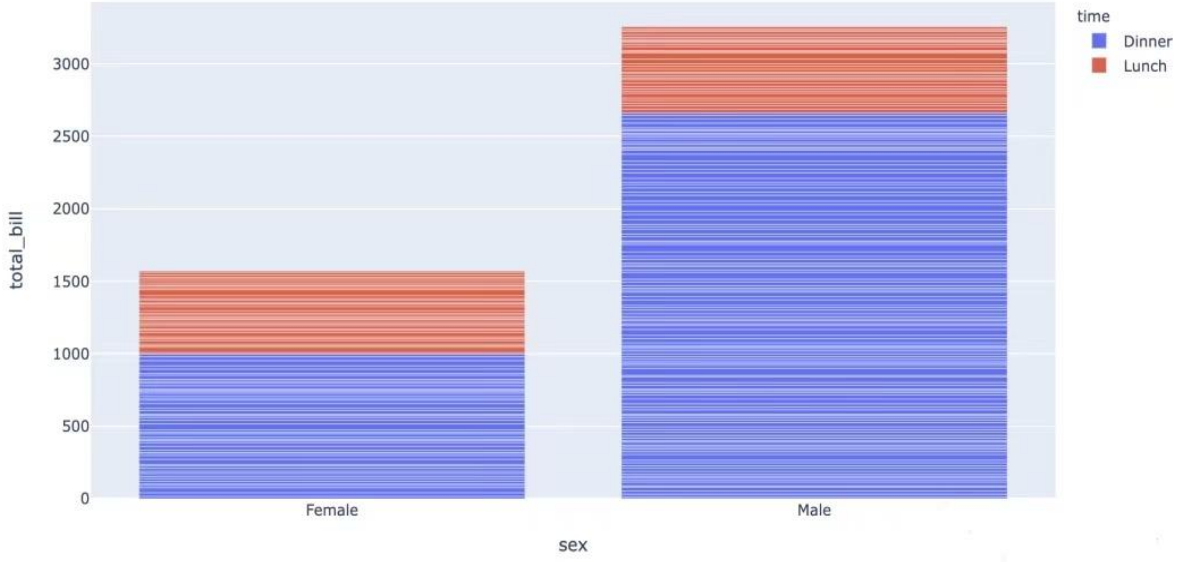
Python > Figure Reference > Single-Page Reference

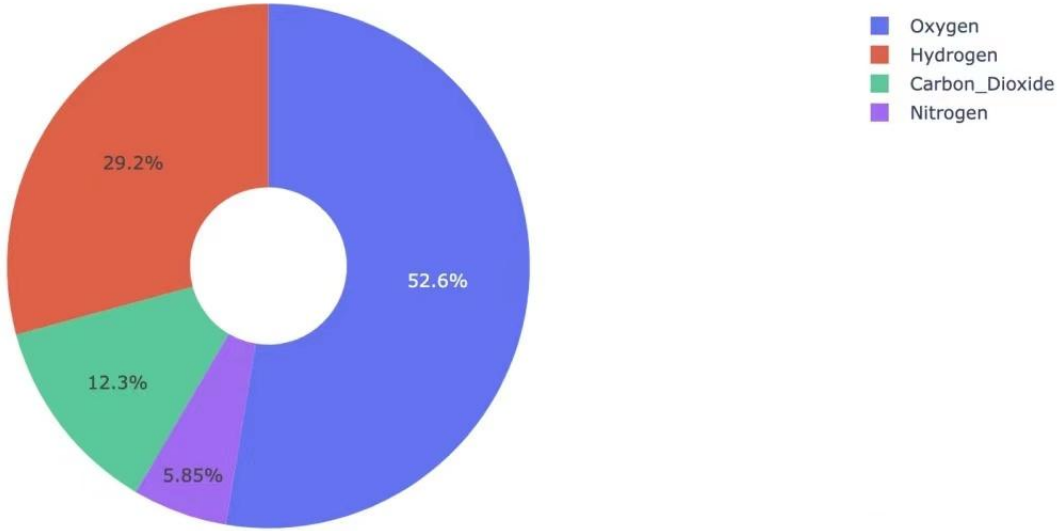
Python Figure Reference: Single-Page

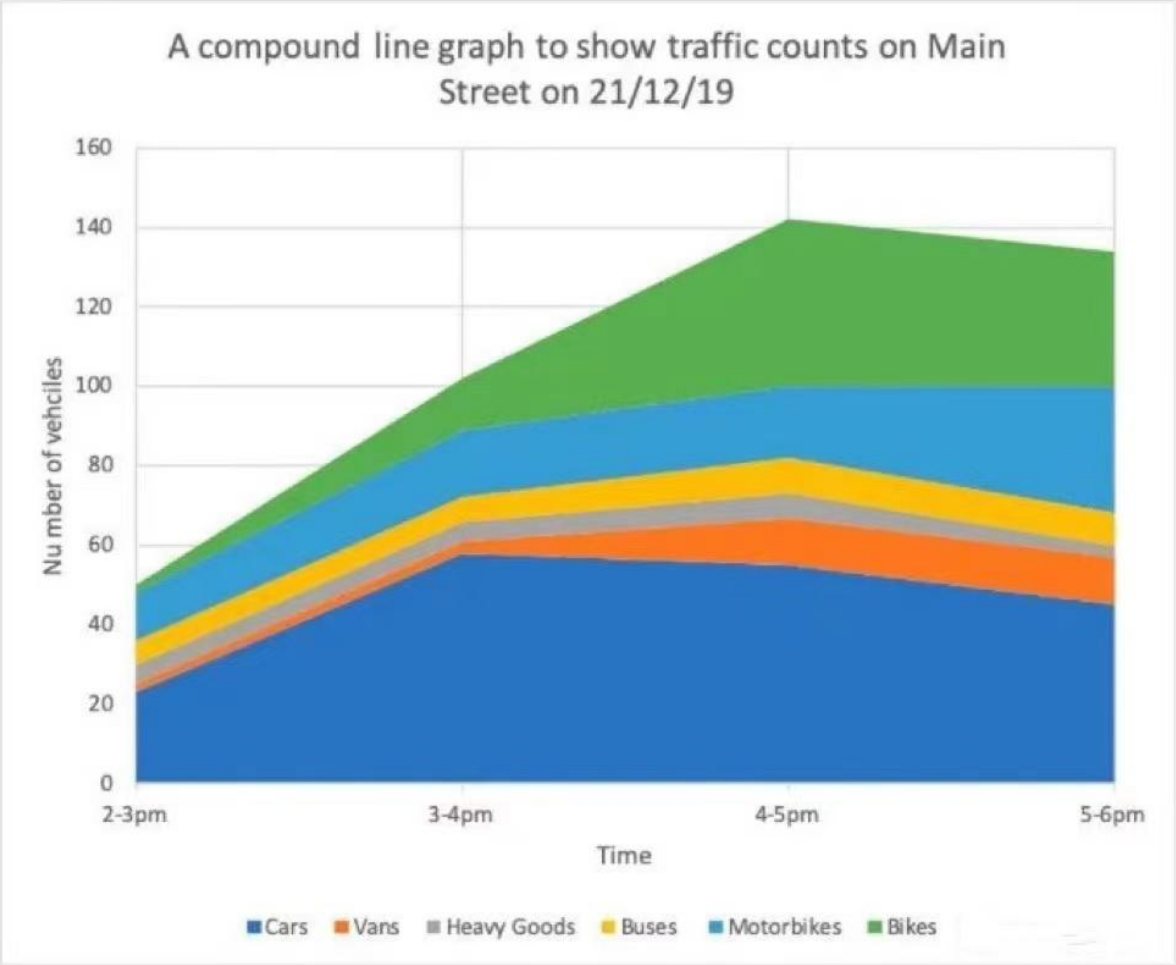
This page is the exhaustive reference for all of the attributes in the core figure data structure that the `plotly` library operates on. It is automatically-generated from the machine-readable Plotly.js schema reference.

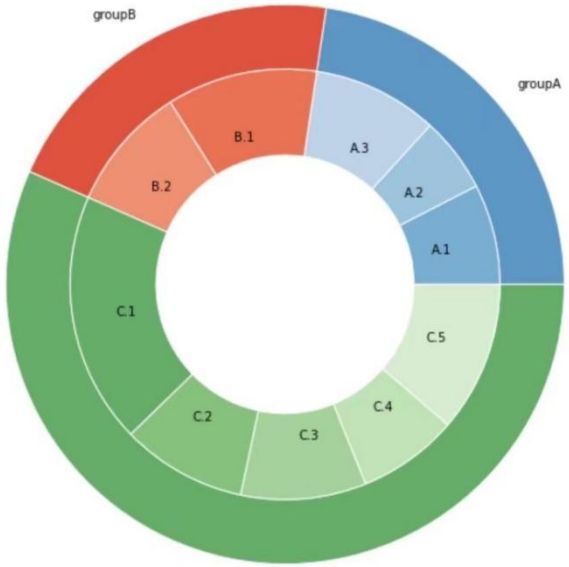
Figures are represented as trees with named nodes called "attributes". The root node of the tree has three top-level attributes: `data`, `layout` and `frames`. Attributes are referred to in text and in this page by their full "path" i.e. the dot-delimited concatenation of their parents. For example `"layout.width"` refers to the attribute whose key is `"width"` inside a dict which is the value associated with a key `"layout"` at the root of the figure. If one of the parents is a list rather than a dict, a set of brackets is inserted in the path when referring to the attribute in the abstract, e.g. `"layout.annotations[].text"`. Finally, as explained below, the top-level "data" attribute defines a list of typed objects called "traces" with the schema dependent upon the type, and these attributes' paths are listed in this page as `"data[type=scatter].name"`. When manipulating a `plotly.graph_objects.Figure` object, attributes can be set either directly using Python object attributes e.g. `fig.layout.title.font.family="Open Sans"` or using update methods and "magic underscores" e.g. `fig.update_layout(title_font_family="Open Sans")`

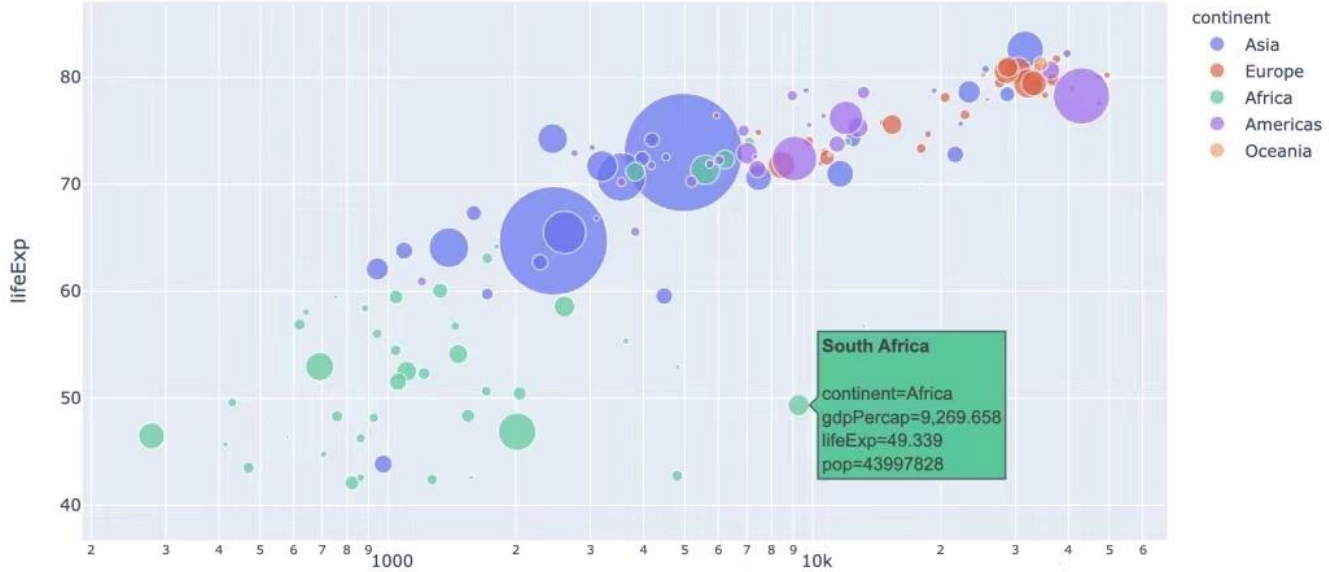
When building a figure, it is *not necessary to populate every attribute* of every object. At render-time, the JavaScript layer will compute default values for each required unspecified attribute, depending upon the ones that are specified, as documented in this page. An example of this would be `layout.xaxis.range`, which may be specified explicitly, but if not will be computed based on the range of `x` values for every trace linked to that axis. The JavaScript layer will ignore unknown attributes or malformed values, although the `plotly.graph_objects` module provides Python-side validation for attribute values. Note also that if the `layout.template` key is present (as it is by default) then default

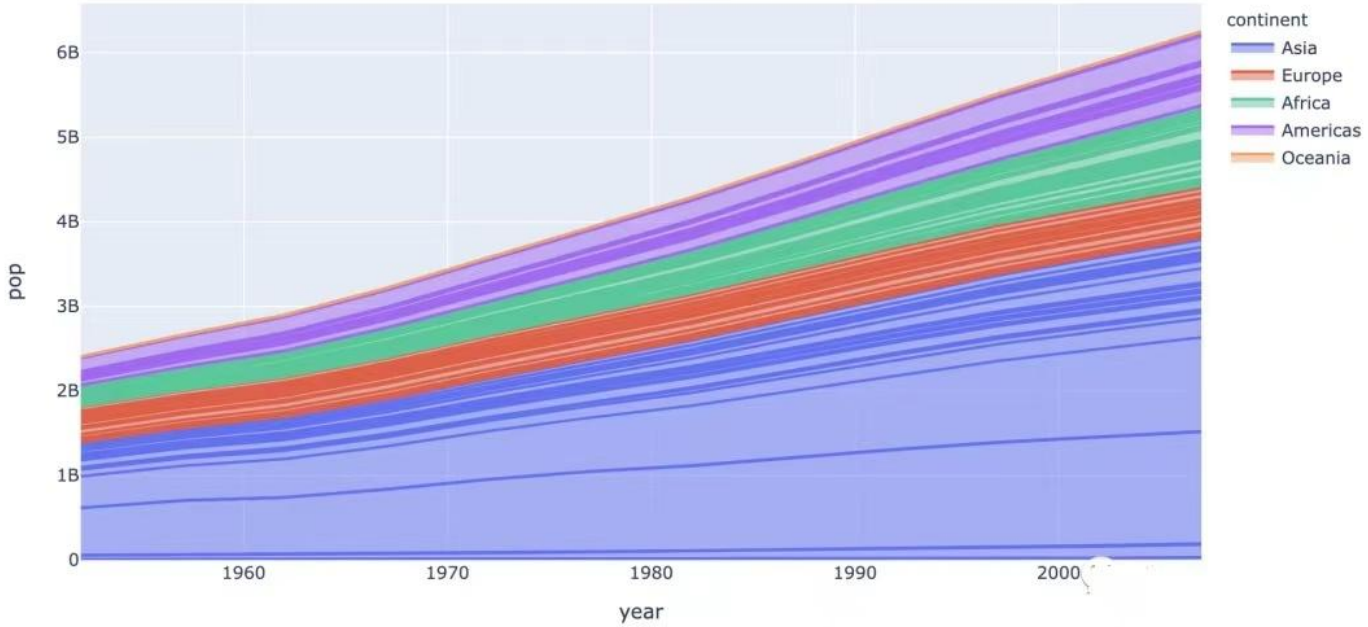

time
Dinner
Lunch
total_bill
3000
2500
2000
1500
1000
500
0
Female
Male
sex

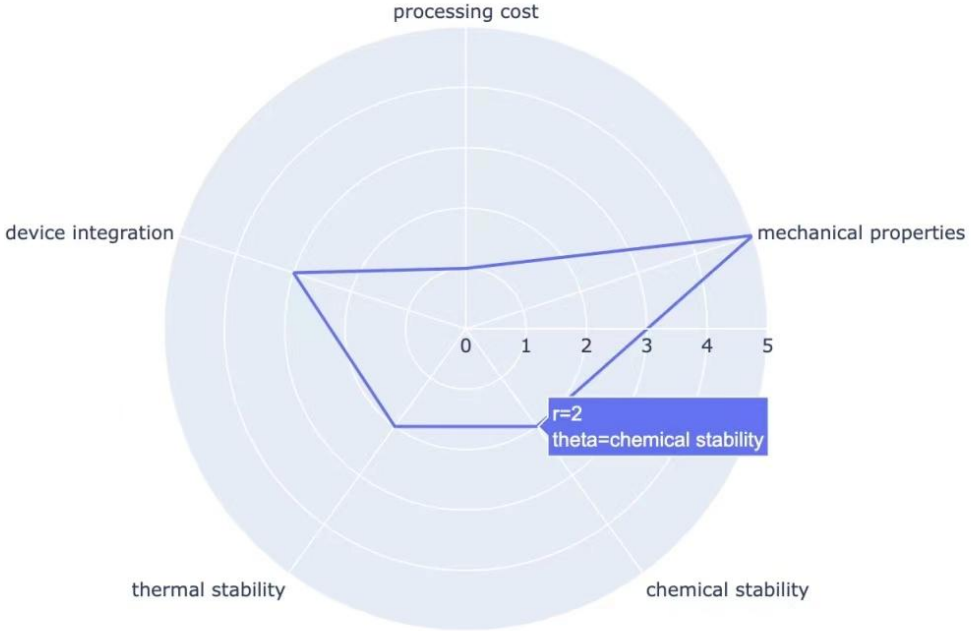

Oxygen
Hydrogen
Carbon_Dioxide
Nitrogen
29.2%
52.6%
12.3%
5.85%


A compound line graph to show traffic counts on Main Street on 21/12/19
Nu mber of vehciles
160
140
120
100
80
60
40
20
0
2-3pm
3-4pm
4-5pm
5-6pm
Time
Cars
Vans
Heavy Goods
Buses
Motorbikes
Bikes


groupB
groupA
B.1
B.2
A.3
A.2
A.1
C.1
C.5
C.2
C.3
C.4


continent
Asia
Europe
Africa
Americas
Oceania
lifeExp
80
70
60
50
40
1000
10k
South Africa
continent=Africa
gdpPercap=9,269.658
lifeExp=49.339
pop=43997828


continent
Asia
Europe
Africa
Americas
Oceania
pop
6B
5B
4B
3B
2B
1B
0
1960
1970
1980
1990
2000
year


processing cost
mechanical properties
device integration
0
1
2
3
4
5
r=2
theta=chemical stability
thermal stability
chemical stability

Types and References

Plot type	plotly	seaborn
Simple bar graph	express bar	barplot
Grouped bar graph	color attribute and barmode='group'	hue attribute
Stacked bar graph	color attribute	label and color attributes with multiple plots
Simple line graph	express line	lineplot
Multiple line graph	color and symbol attributes	hue attribute
Simple pie chart	express pie	matplotlib.pyplot.pie
Exploded pie chart	graph_objects Pie with pull attribute	explode attribute
Donut chart	graph_objects Pie with hole attribute	Add matplotlib.pyplot.Circle
3D pie chart	Use pygooglechart package	shadow attribute
Normal histogram	express histogram	histplot
Bimodal histogram	color attribute	kdeplot
Area chart	express area	matplotlib.pyplot.stackplot

Dot graph	express scatter	stripplot
Scatter plot	express scatter	scatterplot
Bubble chart	express scatter with color and size attributes	scatterplot with size attribute
Radar chart	express line_polar	matplotlib.pyplot figure
Pictogram	graph_objects Figure having Scatter with marker attribute	matplotlib.pyplot figure with pywaffle package
Spline chart	express line with line_shape='spline'	Scipy.interpolate.make_interp_spline
Box plot	express box	boxplot
Violin Plot	express violin	violinplot

Install plotly

- 启动Anaconda Prompt来安装plotly
- 输入‘pip install plotly’
- 并启动‘jupyter notebook’

Anaconda Prompt - jupyter notebook

```
(base) C:\Users\lychee>pip install plotly
Requirement already satisfied: plotly in c:\users\lychee\anaconda3\lib\site-packages (4.1.1)
Requirement already satisfied: retrying>=1.3.3 in c:\users\lychee\anaconda3\lib\site-packages (from plotly) (1.3.3)
Requirement already satisfied: six in c:\users\lychee\anaconda3\lib\site-packages (from plotly) (1.11.0)

(base) C:\Users\lychee>jupyter notebook
[I 16:33:04.018 NotebookApp] The port 8888 is already in use, trying another port.
[I 16:33:04.120 NotebookApp] JupyterLab beta preview extension loaded from C:\Users\lychee\Anaconda3\lib\site-packages\j
upyterlab
[I 16:33:04.127 NotebookApp] JupyterLab application directory is C:\Users\lychee\Anaconda3\share\jupyter\lab
[I 16:33:04.239 NotebookApp] Serving notebooks from local directory: C:\Users\lychee
[I 16:33:04.239 NotebookApp] 0 active kernels
[I 16:33:04.241 NotebookApp] The Jupyter Notebook is running at:
[I 16:33:04.241 NotebookApp] http://localhost:8889/?token=e2e04f31b51ee6c2c2f01a0d0157fbea4fffe1f8e7d7e944
[I 16:33:04.242 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 16:33:04.243 NotebookApp]
```

Install plotly

```
In [ ]: import plotly

In [ ]: pip install plotly

In [ ]: conda install plotly
```

```
In [1]: import plotly

In [2]: plotly.__version__

Out[2]: '4.1.1'

In [ ]:
```

Hands-on Practice

```
In [9]: import pandas as pd
```

```
In [10]: data = pd.read_csv('data/nz_weather.csv')
```

```
In [11]: data.head()
```

Out[11]:

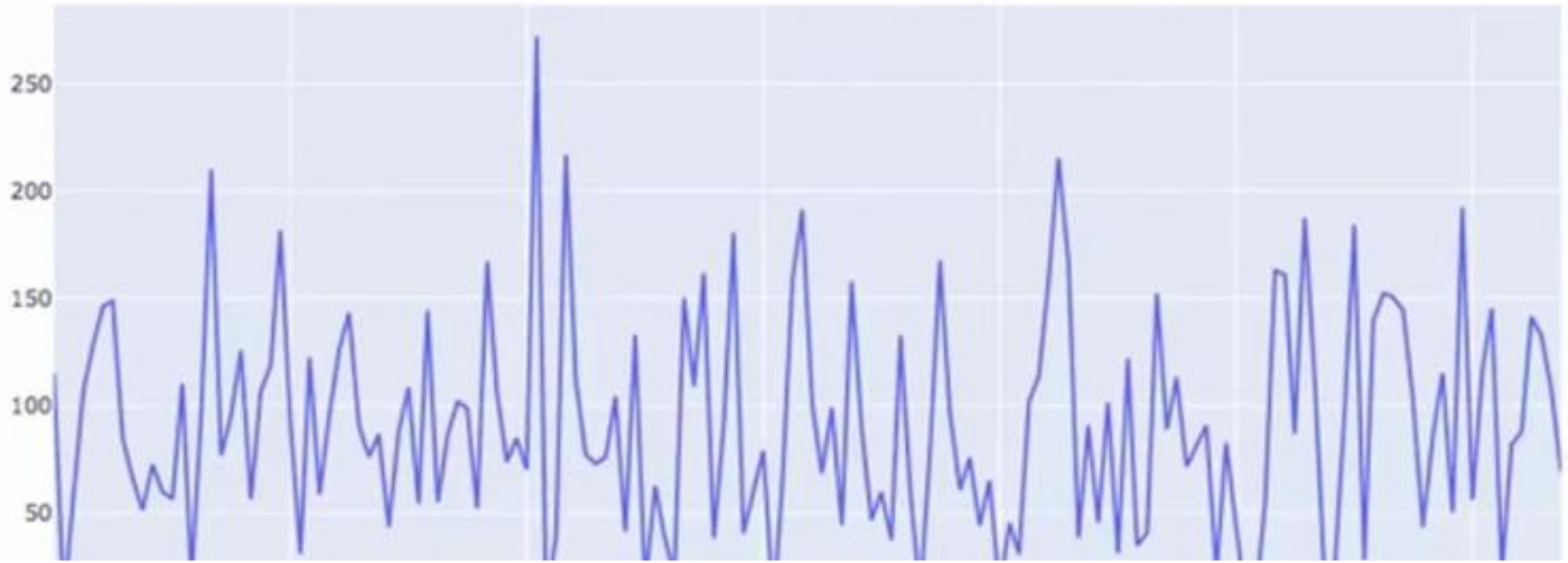
	DATE	Auckland	Christchurch	Dunedin	Hamilton	Wellington
0	2000-01	115.4	47.2	174.8	96.2	91.8
1	2000-02	8.4	25.2	41	8.2	35.2
2	2000-03	57.2	60.8	74.2	33.8	53.4
3	2000-04	106.8	58.2	50	129.6	109.8
4	2000-05	128.2	62.0	'-	98.2	78.2

```
In [18]: data['Auckland'], data[]
```

```
Out[18]: 0      115.4
         1        8.4
         2       57.2
         3      106.8
         4      128.2
         5      146.2
         6      148.8
         7       84.6
         8       66.0
```

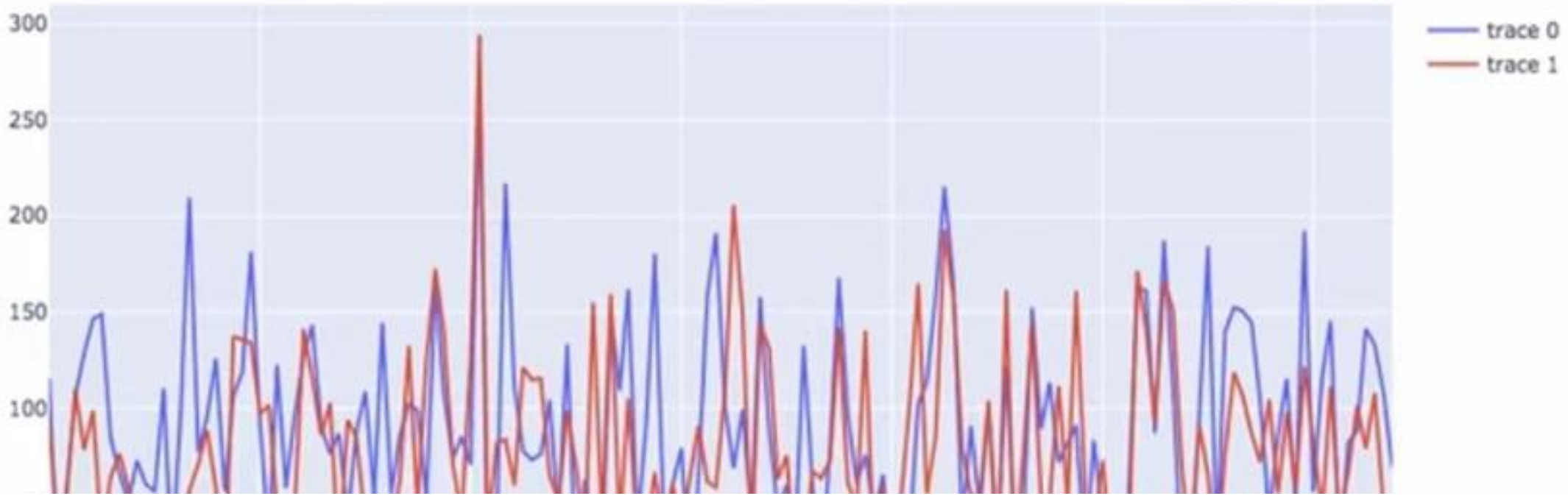
Scatter

```
In [21]: line = go.Scatter(x=data['DATE'], y=data['Auckland'])
         fig  = go.Figure(line)
         fig.show()
```

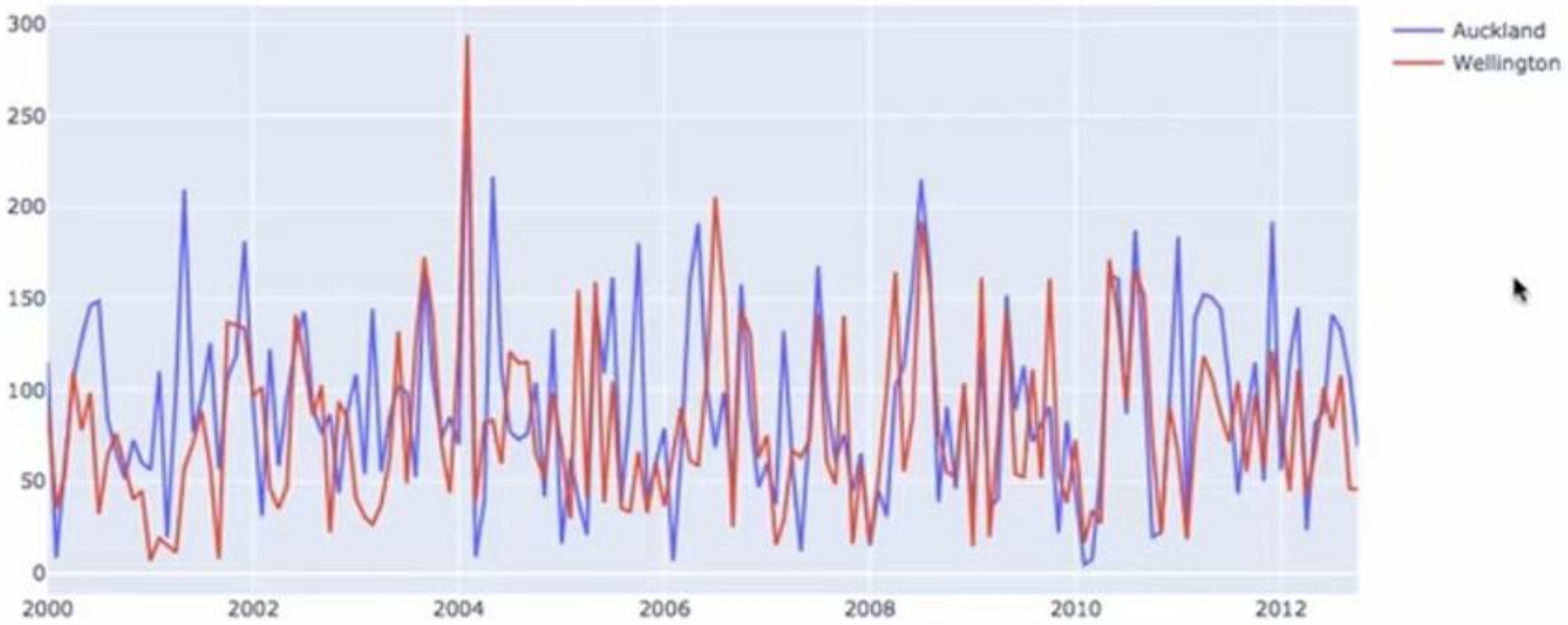


Scatter

```
In [22]: line1 = go.Scatter(x=data['DATE'], y=data['Auckland'], name='Auckland')
         line2 = go.Scatter(x=data['DATE'], y=data['Wellington'], name='Wellington')
         fig  = go.Figure([line1, line2])
         fig.show()
```



Scatter

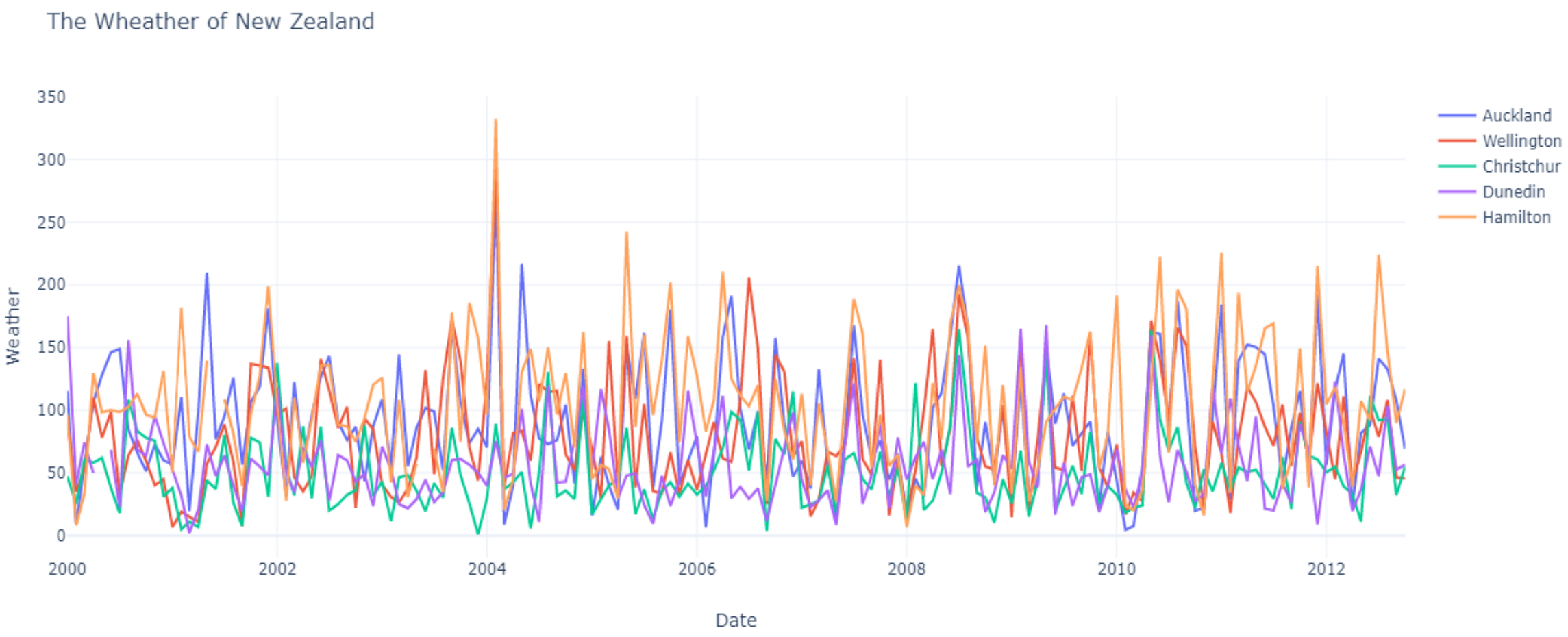

300
250
200
150
100
50
0
2000
2002
2004
2006
2008
2010
2012
Auckland
Wellington

Scatter

```python
line1 = go.Scatter(x=data['DATE'], y=data['Auckland'], name='Auckland')
line2 = go.Scatter(x=data['DATE'], y=data['Wellington'], name='Wellington')
line3 = go.Scatter (x=data['DATE'], y=data['Christchurch'], name='Christchur')
line4= go.Scatter(x=data['DATE'],y=data['Dunedin'],name='Dunedin')

line5=go.Scatter(x=data['DATE'],y=data['Hamilton'],name='Hamilton')
fig  = go.Figure([line1, line2,line3,line4,line5])

# mzy新加的两行代码展示散点图（ctrl+/）
# date_and_adress_scatter_line = go.Scatter(x=data['DATE'],y=data['Hamilton'],name='Hamilton',mode="markers")
# fig  = go.Figure([date_and_adress_scatter_line])



fig.update_layout(
    title = "The Wheather of New Zealand",
    xaxis_title = "Date",
    yaxis_title = "Weather"
)
fig.show()
```

✓ 0.2s

Scatter


The Wheather of New Zealand
350
300
250
200
150
100
50
0
Weather
2000
2002
2004
2006
2008
2010
2012
Date
Auckland
Wellington
Christchur
Dunedin
Hamilton

Bar

```
data_2010 = data[(data['DATE'] >= '2010-01') & (data['DATE'] < '2011-01')]
data_2010
```

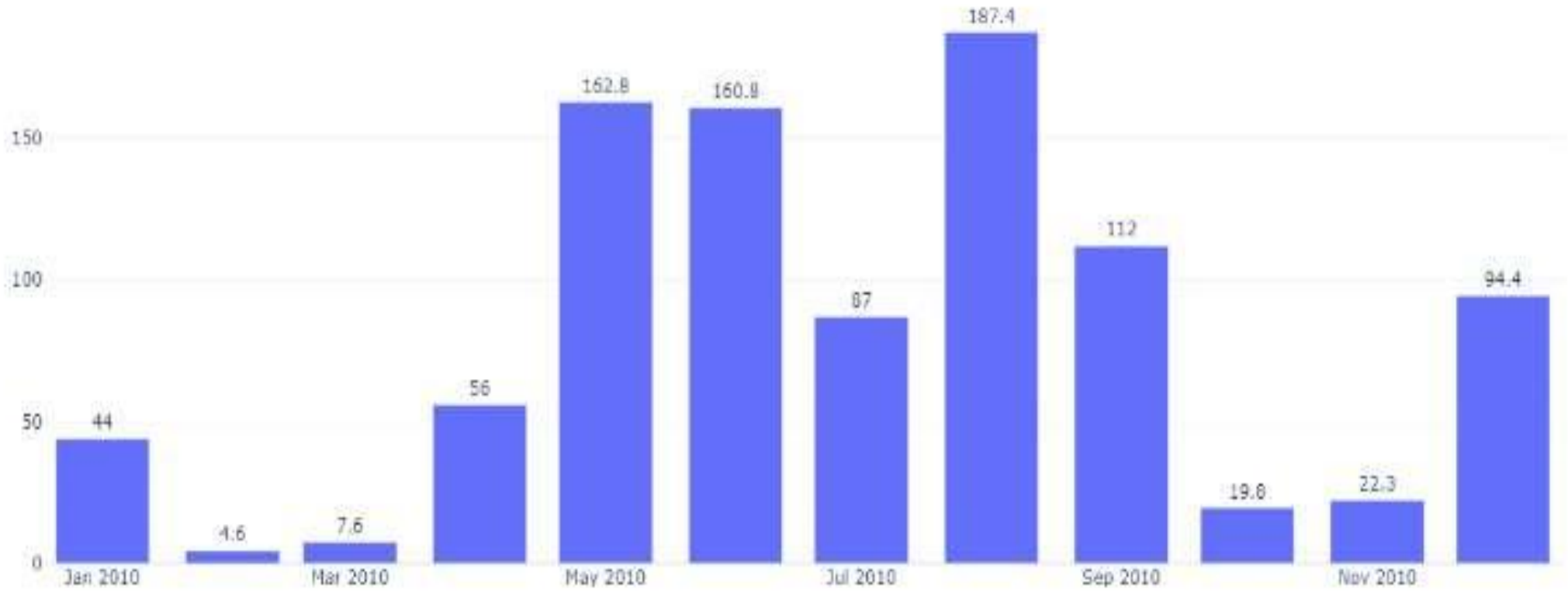
[8] ✓ 0.6s

	DATE	Auckland	Christchurch	Dunedin	Hamilton	Wellington
120	2010-01	44.0	32.6	69	191.6	72.8
121	2010-02	4.6	18.2	36.6	24.4	16.8
122	2010-03	7.6	22.4	22.6	20.2	34.2
123	2010-04	56.0	24.0	50.4	36.6	27.2
124	2010-05	162.8	163.8	159.8	120.4	171.4
125	2010-06	160.8	93.1	63.6	222.6	140.3
126	2010-07	87.0	67.6	26.6	65.8	92.4
127	2010-08	187.4	86.4	68.2	196.1	165.8
128	2010-09	112.0	42.0	51	180.8	151.6
129	2010-10	19.8	22.0	27.6	41.2	71.8
130	2010-11	22.3	53.0	32.8	16	21.4
131	2010-12	94.4	35.2	119.8	120.2	91.0

```
bar1 = go.Bar(x = data_2010['DATE'], y = data_2010['Auckland'],
              text = data_2010['Auckland'], textposition='outside')
fig = go.Figure(bar1)
fig.show()
```

[9] ✓ 0.2s

Bar


150
100
50
0
44
4.6
7.6
56
162.8
160.8
87
187.4
112
19.8
22.3
94.4
Jan 2010
Mar 2010
May 2010
Jul 2010
Sep 2010
Nov 2010

Bar

```
data_2010 = data[(data['DATE'] >= '2010-01') & (data['DATE'] < '2011-01')]
data_2010
```

[8] ✓ 0.6s

	DATE	Auckland	Christchurch	Dunedin	Hamilton	Wellington
120	2010-01	44.0	32.6	69	191.6	72.8
121	2010-02	4.6	18.2	36.6	24.4	16.8
122	2010-03	7.6	22.4	22.6	20.2	34.2
123	2010-04	56.0	24.0	50.4	36.6	27.2
124	2010-05	162.8	163.8	159.8	120.4	171.4
125	2010-06	160.8	93.1	63.6	222.6	140.3
126	2010-07	87.0	67.6	26.6	65.8	92.4
127	2010-08	187.4	86.4	68.2	196.1	165.8
128	2010-09	112.0	42.0	51	180.8	151.6
129	2010-10	19.8	22.0	27.6	41.2	71.8
130	2010-11	22.3	53.0	32.8	16	21.4
131	2010-12	94.4	35.2	119.8	120.2	91.0

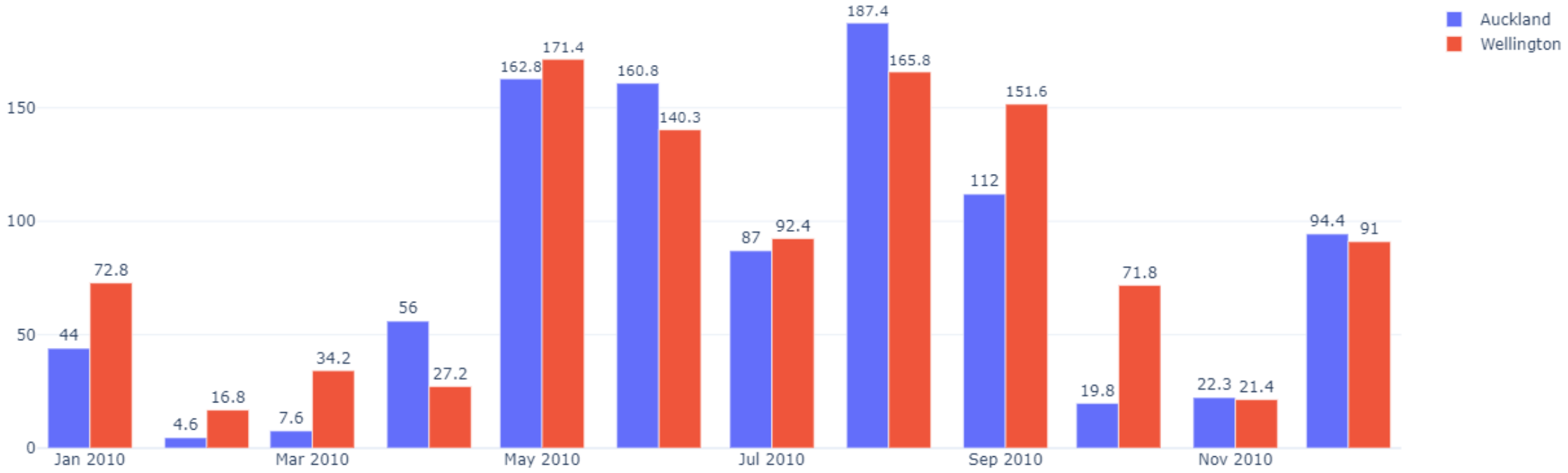
```
bar1 = go.Bar(x = data_2010['DATE'], y = data_2010['Auckland'],
              text = data_2010['Auckland'], textposition='outside')
fig = go.Figure(bar1)
fig.show()
```

[9] ✓ 0.2s

Bar

```
bar1 = go.Bar(x = data_2010['DATE'], y = data_2010['Auckland'],
              text = data_2010['Auckland'], textposition='outside', name='Auckland')
bar2 = go.Bar(x = data_2010['DATE'], y = data_2010['Wellington'],
              text = data_2010['Wellington'], textposition='outside', name='Wellington')
fig = go.Figure([bar1, bar2])
fig.show()
```

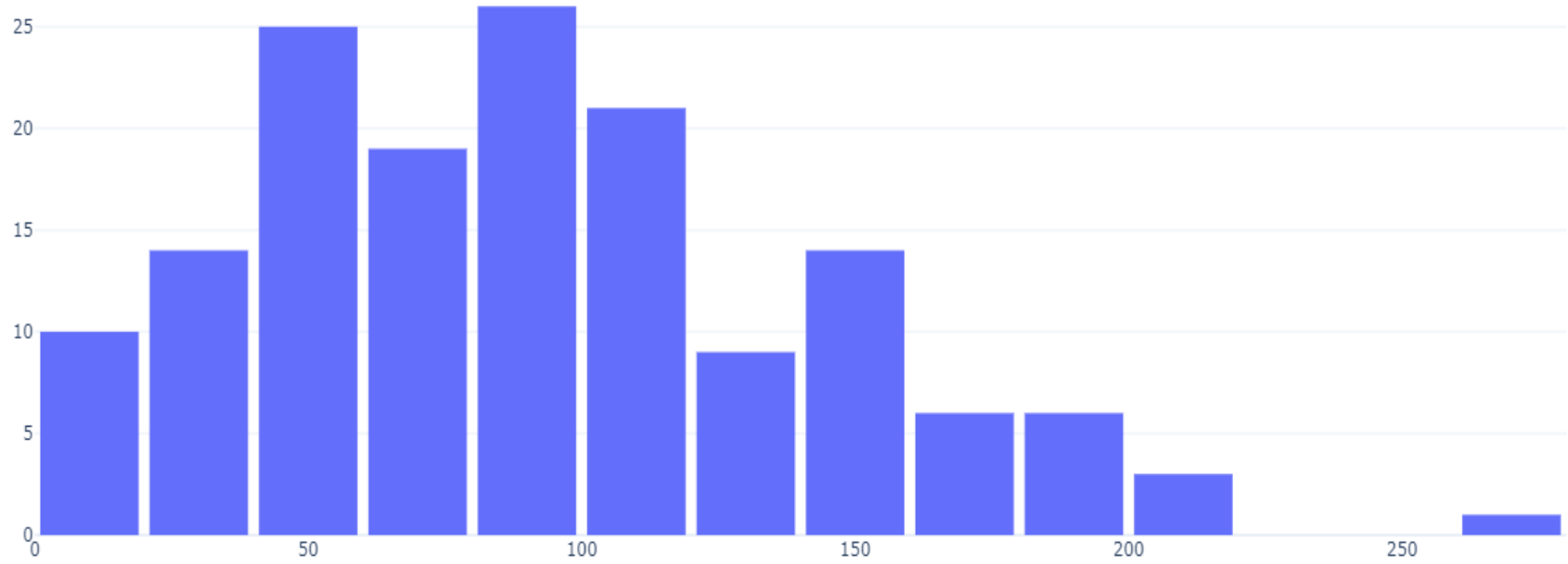
✓ 0.1s



Histogram

```python
hist = go.Histogram(x = data['Auckland'])
fig = go.Figure(hist)
fig.update_layout(bargap=0.1)
fig.show()
```

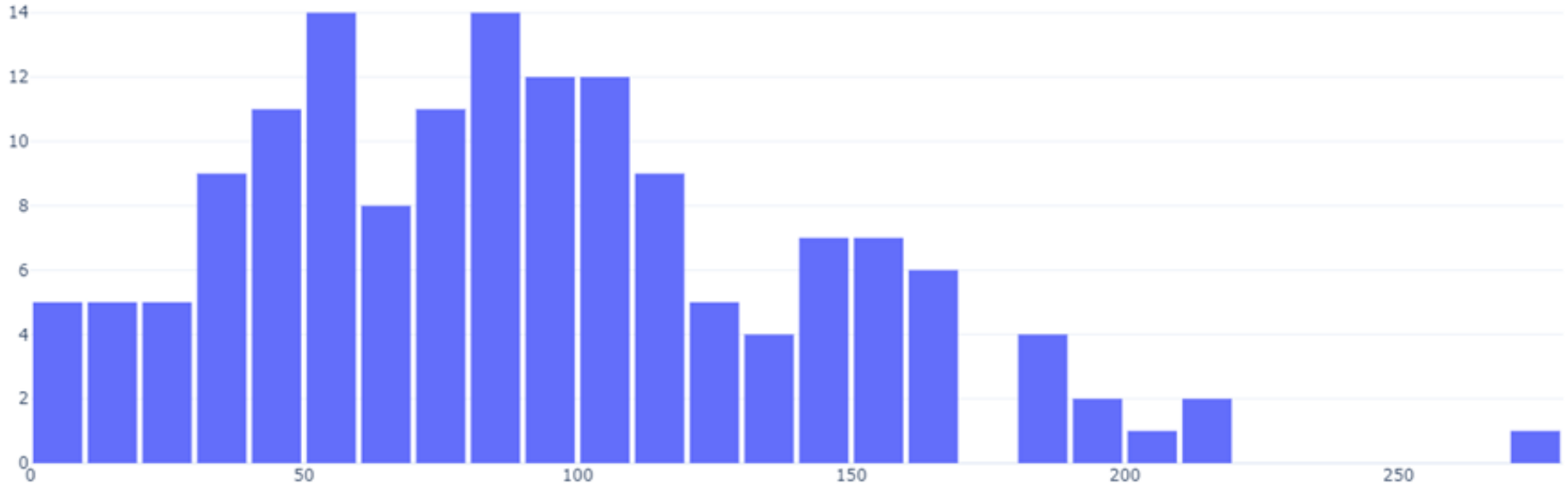
✓ 0.1s



Histogram

```python
hist = go.Histogram(x = data['Auckland'], xbins={'size': 10})
fig = go.Figure(hist)
fig.update_layout(bargap=0.1)
fig.show()
```

✓ 0.2s



Thanks for listening!